

■ 3.5.4 Advanced Topic: The Representation of Derivatives

Derivatives in *Mathematica* work essentially the same as in standard mathematics. The usual mathematical notation, however, often hides many details. To understand how derivatives are represented in *Mathematica*, we must look at these details.

The standard mathematical notation $f'(0)$ is really a short hand for $\frac{d}{dt}f(t)|_{t=0}$, where t is a “dummy variable”. Similarly, $f'(x^2)$ is a short hand for $\frac{d}{dt}f(t)|_{t=x^2}$. As suggested by the notation f' , the object $\frac{d}{dt}f(t)$ can in fact be viewed as a “pure function”, to be evaluated with a particular choice of its parameter t . You can in fact think of the operation of differentiation as acting on a *function* f , to give a new *function*, usually called f' .

With functions of more than one argument, the simple notation based on primes breaks down. You cannot tell for example whether $g'(0,1)$ stands for $\frac{d}{dt}g(t,1)|_{t=0}$ or $\frac{d}{dt}g(0,t)|_{t=1}$, and for almost any g , these will have totally different values. Once again, however, t is just a dummy variable, whose sole purpose is to show with respect to which “slot” g is to be differentiated.

In *Mathematica*, as in some branches of mathematics, it is convenient to think about a kind of differentiation that acts on *functions*, rather than expressions. We need an operation that takes the function f , and gives us the *derivative function* f' . Operations such as this that act on *functions*, rather than variables, are known in mathematics as *functionals*.

The object **f'** in *Mathematica* is the result of applying the differentiation functional to the function **f**. The full form of **f'** is in fact **Derivative[1][f]**. **Derivative[1]** is the *Mathematica* differentiation functional.

The arguments in the functional **Derivative**[$n_1, n_2, \dots$] specify how many times to differentiate with respect to each “slot” of the function on which it acts. By using functionals to represent differentiation, *Mathematica* avoids any need to introduce explicit “dummy variables”.

This is the full form of the derivative of the function **f**.

```
In[1]:= f' // FullForm
Out[1]//FullForm= Derivative[1][f]
```

Here an argument **x** is supplied.

```
In[2]:= f'[x] // FullForm
Out[2]//FullForm= Derivative[1][f][x]
```

This is the second derivative.

```
In[3]:= f''[x] // FullForm
Out[3]//FullForm= Derivative[2][f][x]
```

This gives a derivative of the function g with respect to its second “slot”.

```
In[4]:= D[g[x, y], y]
Out[4]=  $g^{(0,1)}[x, y]$ 
```

Here is the full form.

```
In[5]:= % // FullForm
Out[5]//FullForm= Derivative[0, 1][g][x, y]
```

Here is the second derivative with respect to the variable y , which appears in the second slot of g .

```
In[6]:= D[g[x, y], {y, 2}] // FullForm
Out[6]//FullForm= Derivative[0, 2][g][x, y]
```

This is a mixed derivative.

```
In[7]:= D[g[x, y], x, y, y] // FullForm
Out[7]//FullForm= Derivative[1, 2][g][x, y]
```

Since `Derivative` only specifies how many times to differentiate with respect to each slot, the order of the derivatives is irrelevant.

```
In[8]:= D[g[x, y], y, y, x] // FullForm
Out[8]//FullForm= Derivative[1, 2][g][x, y]
```

Here is a more complicated case, in which both arguments of g depend on the differentiation variable.

```
In[9]:= D[g[x, x], x]
Out[9]=  $g^{(0,1)}[x, x] + g^{(1,0)}[x, x]$ 
```

This is the full form of the result.

```
In[10]:= % // FullForm
Out[10]//FullForm=
  Plus[Derivative[0, 1][g][x, x],
    Derivative[1, 0][g][x, x]]
```

The object f' behaves essentially like any other function in *Mathematica*. You can evaluate the function with any argument, and you can use standard *Mathematica* `/.` operations to change the argument. (This would not be possible if explicit dummy variables had been introduced in the course of the differentiation.)

This is the *Mathematica* representation of the derivative of a function f , evaluated at the origin.

```
In[11]:= f'[0] // FullForm
Out[11]//FullForm= Derivative[1][f][0]
```

The result of this derivative involves f' evaluated with the argument x^2 .

```
In[12]:= D[f[x^2], x]
Out[12]= 2 x f'[x^2]
```

You can evaluate the result at the point $x = 2$ by using the standard *Mathematica* replacement operation.

```
In[13]:= % /. x->2
Out[13]= 4 f'[4]
```

There is some slight subtlety when you need to deduce the value of f' based on definitions for objects like $f[x_]$.

Here is a definition for a function h . `In[14]:= h[x_] := x^4`

When you take the derivative of $h[x]$, `In[15]:= D[h[x], x]`
 $Mathematica$ first evaluates $h[x]$, then
`Out[15]= 4 x^3`
differentiates the result.

You can get the same result by applying
the function h' to the argument x . `In[16]:= h'[x]`
`Out[16]= 4 x^3`

Here is the function h' on its own. `In[17]:= h'`
`Out[17]= 4 #1^3 &`

The function f' is completely determined by the form of the function f . Definitions for objects like $f[x_]$ do not immediately apply however to expressions like $f'[x_]$. The problem is that $f'[x]$ has the full form `Derivative[1][f][x]`, which nowhere contains anything that explicitly matches the pattern $f[x_]$. In addition, for many purposes it is convenient to have a representation of the function f' itself, without necessarily applying it to any arguments.

What $Mathematica$ does is to try and find the explicit form of a *pure function* which represents the object f' . When $Mathematica$ gets an expression like `Derivative[1][f]`, it effectively converts it to the explicit form `D[f[#], #]&` and then tries to evaluate the derivative. In the explicit form, $Mathematica$ can immediately use values that have been defined for objects like $f[x_]$. If $Mathematica$ succeeds in doing the derivative, it returns the explicit pure function result. If it does not succeed, it leaves the derivative in the original f' form.

This gives the derivative of $\tan$ in pure
function form. `In[18]:= Tan'`
`Out[18]= Cos[#1]^-2 &`

Here is the result of applying the pure
function to the specific argument y . `In[19]:= % [y]`
`Out[19]= Cos[y]^-2`